Informe — Trabajo Práctico N.º 3 Análisis y Diseño de Algoritmos

Tema: Transformar y Conquistar — estructuras avanzadas y algoritmos de ordenamiento sobre 100 millones de datos

Apellido y Nombre: Mendoza Jeremias ·

Institución: Universidad Nacional de Villa Mercedes · 2026

Repositorio: https://github.com/jeremendoza01/tp3

Índice

1. Resumen ejecutivo
2. Entorno de pruebas y metodología
3. Descripción del programa
4. Resultados y análisis por consigna
5. Consigna l) Comparación de estructuras y algoritmos
6. Conclusiones
7. Limitaciones y posibles mejoras
8. Instrucciones de ejecución
9. Anexo: capturas de pantalla

1. Resumen ejecutivo

Se desarrolló en Python un programa con menú interactivo de 10 opciones que resuelve las consignas a) a k). Este informe documenta el diseño, los resultados medidos en un equipo real y el análisis comparativo de la consigna l). Los hallazgos principales son:

Hallazgo	Evidencia medida
Counting Sort ordenó los 100.000.000 de datos completos	71,03 s (≈ 0,71 µs por elemento), a costa de ≈ 800 MB para el arreglo de conteo
Selection Sort es inviable para grandes volúmenes	157,25 s con solo 100.000 elementos; extrapolación a 100 M: ≈ 5 años
Heap Sort escala como n log n	5,17 s (1 M) → 58,36 s (10 M): factor 11,29 observado contra 11,67 teórico
En búsqueda, el AVL supera ampliamente al Heap	22 µs contra 0,427 s: ≈ 19.400 veces (≈ 1.900 veces a igual n)
En inserción y extracción de prioridad, el Heap es más eficiente	≈ 0,77 µs por inserción (construcción) contra 9,84 µs en el AVL
Los tres archivos ordenados fueron verificados	Resultado ORDENADO en los tres casos

2. Entorno de pruebas y metodología

2.1 Hardware y software

Componente	Especificación
Procesador	AMD Ryzen 5 5600GT with Radeon Graphics, 3,60 GHz

Componente	Especificación
Memoria RAM	16,0 GB (15,4 GB utilizables)
Almacenamiento	489 GB usados de 943 GB
Sistema operativo	Windows de 64 bits, procesador x64
Lenguaje	Python 3. (intérprete CPython)

2.2 Metodología de medición

- Cada medición corresponde a una corrida, tomada con time.time(). Los tiempos incluyen el costo de entrada/salida (lectura de líneas, conversión a entero, escritura), salvo que se indique lo contrario.
- Los archivos se leen en modo streaming, línea a línea, para no cargar 100 millones de datos en memoria cuando no es necesario.
- Los tiempos de operaciones individuales (búsqueda, inserción, eliminación) son del orden de microsegundos y provienen de una sola ejecución; deben interpretarse como órdenes de magnitud y no como valores exactos.
- Python es un lenguaje interpretado: sus constantes son mayores que las de C. Por eso el análisis se centra en el crecimiento asintótico y en las proporciones entre algoritmos, no en el valor absoluto de los segundos.

2.3 Tamaños de muestra y justificación

La consigna fija 100.000.000 de datos. Para los algoritmos y estructuras cuyo costo o memoria lo hacen impracticable se trabajó con muestras tomadas de los primeros elementos del archivo, indicando el motivo:

Opción	n utilizado	Motivo
Generación, máx/mín, frecuencias, Counting Sort	100.000.000	Costo lineal; se procesa en streaming
Selection Sort	100.000	Costo cuadrático: con 100 M tardaría ≈ 5 años
Árbol AVL	1.000.000	Cada nodo es un objeto con punteros: con 100 M superaría la RAM disponible
Max Heap	10.000.000	Arreglo contiguo, más compacto que el AVL
Heap Sort	1.000.000 y 10.000.000	Dos corridas para verificar el crecimiento n log n

3. Descripción del programa

Opción	Consigna	Función	Complejidad temporal
1	a	generar_archivo_datos	O(n)
2	b, c	buscar_max_min_y_frecuencia	O(n)
3	d	ordenar_counting_sort	O(n + k)
4	e	ordenar_selection_sort	$O(n^2)$
5	i	operar_arbol_avl	Construcción O(n log n); búsqueda y eliminación O(log n)
6	j	operar_heap	Inserción y extracción O(log n); búsqueda O(n)
7	k	ordenar_heap_sort	O(n log n)
8	f, k	verificar_orden	O(n)

Decisiones de diseño

- random.randint(0, 100000000) es inclusivo en ambos extremos, por lo que hay k = 100.000.001 valores posibles.
- La generación escribe en bloques de 1.000.000 de números con una única llamada de escritura por bloque, para reducir el costo de I/O.
- Las opciones b y c se resuelven en un solo recorrido del archivo: se actualizan máximo, mínimo y un Counter (tabla hash) en la misma pasada. Por eso el tiempo de 67,84 s corresponde a ambas consignas.
- Todas las opciones comprueban que Datos.txt exista antes de operar.
- Las operaciones con estructuras reciben el tamaño de muestra por teclado, con un valor por defecto seguro.
- En el AVL, los valores repetidos se insertan en el subárbol derecho. El Heap y el AVL son implementaciones propias; no se usaron librerías de estructuras.
- heapify es recursivo, pero su profundidad máxima es log2(n) (≈ 20 a 24 niveles), por lo que no hay riesgo de desbordar la pila.

4. Resultados y análisis por consigna

4.1 Consigna a) Generación del archivo de datos

Se generaron 100.000.000 de números aleatorios en el rango [0, 100.000.000], uno por línea, en el archivo Datos.txt.

- **Tiempo:** 36,77 s, es decir ≈ 0,37 µs por número.
- **Tamaño estimado del archivo:** con 7,89 dígitos en promedio por número más el salto de línea, ≈ 0,9 a 1,0 GB (estimación; el tamaño exacto figura en las propiedades del archivo).

4.2 Consignas b) y c) Máximo, mínimo y análisis de frecuencia

- **Tiempo del recorrido completo:** 67,84 s (≈ 0,68 µs por línea), incluyendo la actualización de máximo, mínimo y del Counter.
- **Número mayor:** 100.000.000. **Número menor:** 0. Ambos extremos pertenecen al rango y aparecen en el archivo.

Los 10 números más repetidos

Posición	Número	Apariciones
1	65743355	10
2	1888128	10
3	90273931	10
4	36165422	10
5	10347900	10
6	48025495	10
7	73546229	10
8	80313872	10
9	5968729	10
10	13331351	10

Los diez números empatan en 10 apariciones. En caso de empate, Counter.most_common devuelve los elementos en el orden en que aparecieron por primera vez en el archivo, de modo que esta lista es un subconjunto de los valores con 10 repeticiones.

Validación estadística (distribución de Poisson)

Si se generan n = 10^8 valores uniformes sobre k ≈ 10^8 posibilidades, la cantidad de veces que aparece un valor dado sigue (aproximadamente) una distribución de Poisson de media λ = n/k = 1. La cantidad esperada de valores distintos que aparecen exactamente j veces es N · e^(-1) / j!:

Apariciones j	Valores esperados	Apariciones j	Valores esperados
0	36.787.944	7	7.299
1	36.787.944	8	912
2	18.393.972	9	101
3	6.131.324	10	10,1
4	1.532.831	11	0,92
5	306.566	12	0,08
6	51.094		

De aquí se obtiene que se esperan ≈ 11,1 valores con 10 o más apariciones y ≈ 1,0 valores con 11 o más. La probabilidad de que ningún valor supere las 10 apariciones es ≈ e^(-1) ≈ 37 %. Por lo tanto, un máximo de 10 repeticiones, con ≈ 10 valores que lo alcanzan, es coherente con el comportamiento esperado de un generador sin sesgos. Además, aproximadamente el 63,2 % de los valores posibles (≈ 63,2 millones) aparece al menos una vez, lo que justifica que 0 y 100.000.000 estén presentes.

4.3 Consigna d) Ordenamiento por conteo (Counting Sort)

Funcionamiento. (1) Se crea un arreglo conteo de tamaño k + 1 inicializado en cero. (2) Se lee el archivo y, por cada número x, se incrementa conteo[x]. (3) Se recorre conteo de 0 a k y, por cada posición con cantidad c > 0, se escribe c veces el valor en el archivo de salida. Al ordenar enteros usados como clave y dato, no se necesita una fase de acumulación de prefijos y el resultado es trivialmente estable.

Resultados

- **Tiempo total:** 71,03 s sobre 100.000.000 de elementos (≈ 0,71 µs por elemento), con salida en Datos_CountingSort.txt.
- **Memoria:** el arreglo conteo tiene 100.000.001 posiciones; en CPython cada posición es una referencia de 8 bytes, es decir ≈ 800 MB. Los contadores son enteros pequeños (≤ 10), que CPython reutiliza, por lo que no se suman objetos adicionales. Los datos de entrada no se almacenan: se leen en streaming. La memoria depende de k, no de n.
- **Complejidad:** O(n + k) en tiempo y O(k) en espacio. Como aquí k ≈ n, el costo es lineal en el tamaño de la entrada.

Análisis. El tiempo (71,03 s) es del mismo orden que el recorrido de lectura de la opción 2 (67,84 s): el costo está dominado por leer, convertir y escribir 100 millones de líneas, no por la lógica de conteo. Counting Sort no compara elementos, por lo que no está sujeto a la cota inferior Ω(n log n) de los ordenamientos por comparación. Su límite es otro: solo es eficiente si los valores son enteros dentro de un rango acotado y k no es mucho mayor que n.

4.4 Consigna e) Ordenamiento por selección (Selection Sort)

Funcionamiento. Para cada posición i se busca el mínimo del tramo [i, n) y se intercambia con la posición i. Realiza n(n−1)/2 comparaciones y a lo sumo n−1 intercambios, sin importar el orden inicial de los datos: mejor y peor caso son O(n²). Es in-place (O(1) de espacio auxiliar) y no es estable.

Resultados con n = 100.000

- **Tiempo:** 157,25 s (≈ 1,57 ms por elemento), con salida en Datos_SelectionSort.txt.

- **Comparaciones:** ≈ 4,9999 × 10^9; el tiempo medido equivale a ≈ 31 ns por comparación, razonable para un bucle interpretado en Python.

Extrapolación cuadrática ($t \approx 157{,}25\ s \times (n / 100.000)^2$):

n	Tiempo estimado
100.000	157,25 s (medido)
1.000.000	≈ 15.725 s ≈ 4,4 horas
10.000.000	≈ 1,57 × 10^6 s ≈ 18 días
100.000.000	≈ 1,57 × 10^8 s ≈ 5 años

> Esto justifica reducir la muestra a 100.000 elementos y descartar a Selection Sort para grandes volúmenes.

4.5 Consigna k) Ordenamiento por montículo (Heap Sort)

Funcionamiento. (1) *Construcción bottom-up del Max Heap*: se aplica heapify desde el índice n/2 − 1 hasta 0, en O(n). (2) *Fase de extracción*: n − 1 veces se intercambia la raíz (el máximo) con la última posición del heap, se reduce el tamaño del heap en uno y se aplica heapify sobre la raíz, en O(log n) cada vez. El arreglo queda ordenado de forma ascendente, in-place.

Resultados

n	Tiempo	µs por elemento	Archivo
1.000.000	5,17 s	5,17	Datos_HeapSort.txt
10.000.000	58,36 s	5,84	Datos_HeapSort.txt

> **Verificación del orden de crecimiento.** Al multiplicar n por 10, un algoritmo O(n log n) debería multiplicar su tiempo por 10 × log2(10^7) / log2(10^6) ≈ 11,67. El factor observado fue 58,36 / 5,17 ≈ 11,29, muy cercano al teórico y claramente alejado del factor 100 que produciría un algoritmo cuadrático.

> **Proyección a 100 M:** 58,36 s × 10 × (log2 10^8 / log2 10^7) ≈ 667 s ≈ 11 minutos. Es una cota teórica optimista: con 100 M de enteros Python (≈ 3,6 GB) la localidad de caché empeora y el tiempo real sería mayor.

> **Memoria.** Heap Sort es in-place: el espacio auxiliar es O(1), más la pila de recursión de heapify (O(log n), unos 20 a 24 marcos). La lista de datos ocupa, en CPython de 64 bits, ≈ 36 bytes por elemento (8 B de referencia + 28 B del objeto entero): ≈ 36 MB para 1 M y ≈ 360 MB para 10 M (estimación).

4.6 Consigna f) Verificación del ordenamiento

La función verificar_orden recorre cada archivo una sola vez, comparando cada número con el anterior, y se detiene ante la primera violación de a[i] ≤ a[i+1]. Es O(n) en tiempo y O(1) en memoria.

Archivo	Elementos	Resultado	Tiempo
Datos_CountingSort.txt	100.000.000	ORDENADO	17,62 s
Datos_SelectionSort.txt	100.000	ORDENADO	0,02 s
Datos_HeapSort.txt	1.000.000	ORDENADO	0,18 s

> Los tiempos por línea (≈ 0,18, 0,20 y 0,18 µs) son casi idénticos, lo que confirma el costo lineal de la verificación.

4.7 Consigna i) Árbol AVL

Funcionamiento. El AVL es un árbol binario de búsqueda que mantiene en cada nodo la altura de su subárbol y un factor de balance (altura izquierda − altura derecha) en {−1, 0, 1}. Cuando una inserción o eliminación

rompe esa condición se restablece con rotaciones simples (izquierda o derecha) o dobles. Esto garantiza altura O(log n) y, por lo tanto, búsqueda, inserción y eliminación en O(log n).

Resultados con n = 1.000.000

Métrica	Valor
Construcción (1.000.000 inserciones, incluye lectura del archivo)	9,84 s (≈ 9,84 µs por inserción)
Altura del árbol	24
Cantidad total de nodos	1.000.000
Recorrido InOrden (primeros 10 elementos)	172, 198, 323, 412, 414, 429, 483, 538, 697, 859
Búsqueda de 412	ENCONTRADO, 0,000022 s
Eliminación de 859	0,000041 s; nodos restantes: 999.999

Análisis de la altura. Un AVL con n nodos tiene altura máxima 1,44 · log2(n + 2) ≈ 28 para n = 1 M. La altura medida (24) está por debajo de esa cota y equivale a ≈ 1,2 × log2(n), lo que indica un balanceo muy bueno. El InOrden sale en orden ascendente, lo que confirma la propiedad de árbol de búsqueda.

Análisis de las operaciones. La búsqueda recorre a lo sumo ≈ 20 niveles: 22 µs equivalen a ≈ 1,1 µs por nivel en Python. La eliminación (41 µs) es más costosa porque, además de buscar, puede reemplazar por el sucesor inorden y rebalancear en el camino de vuelta de la recursión. El tiempo de inserción se informa como promedio de la construcción; incluye la lectura y conversión de cada línea, por lo que es una cota superior del costo real de insertar.

4.8 Consigna j) Heap binario (Max Heap)

Funcionamiento. El heap es un árbol binario completo almacenado en un arreglo: para el índice i, el padre está en (i−1)//2 y los hijos en 2i+1 y 2i+2. En un Max Heap cada padre es mayor o igual que sus hijos, por lo que la raíz es el máximo. Insertar agrega al final y *sube* el elemento mientras sea mayor que su padre; extraer el máximo mueve el último elemento a la raíz y lo *hunde* (heapify_down). Ambas operaciones son O(log n). No existe orden entre hermanos, por lo que buscar un valor arbitrario es O(n).

Resultados con n = 10.000.000

Métrica	Valor
Construcción (10.000.000 inserciones sucesivas, incluye lectura)	7,70 s (≈ 0,77 µs por inserción)
Cantidad de elementos	10.000.000
Elemento raíz (máximo)	99.999.997
Altura aproximada	23 (= parte entera de log2(10^7))
Inserción de 165	0,000009 s
Búsqueda de 666 (no encontrado)	0,427438 s (≈ 43 ns por elemento recorrido)
Extracción del máximo (99.999.997)	0,000051 s; nueva raíz: 99.999.995

Análisis.

- La inserción de 165 tardó solo 9 µs porque es un valor pequeño que, al ser menor que su padre, no necesita subir ningún nivel: es el mejor caso. El peor caso sería O(log n) ≈ 23 intercambios. En datos aleatorios la inserción es O(1) en promedio, lo que explica que la construcción por inserciones sucesivas tarde solo ≈ 0,77 µs por elemento.
- La búsqueda de un valor ausente obliga a recorrer los 10 millones de celdas: es el peor caso O(n) y confirma que el heap no está pensado para búsqueda.
- La extracción del máximo (51 µs) recorre hasta ≈ 23 niveles, con dos comparaciones por nivel.

- Se implementó el Max Heap con eliminación del máximo. El Min Heap es simétrico: basta invertir las comparaciones, y su eliminación del mínimo tiene el mismo costo O(log n).

5. Consigna I) Comparación de estructuras y algoritmos

5.1 Árbol AVL contra Heap

Operación	AVL (teoría)	Heap (teoría)	AVL (medido, n = 1 M)	Heap (medido, n = 10 M)
Inserción	O(log n)	O(log n) peor caso; O(1) promedio	≈ 9,84 µs (promedio de construcción)	≈ 0,77 µs (promedio de construcción); 9 µs (caso aislado)
Búsqueda	O(log n)	O(n)	22 µs	427.438 µs
Eliminación	O(log n), cualquier valor	O(log n) solo la raíz; un valor arbitrario requiere O(n) de búsqueda	41 µs	51 µs (máximo)
Memoria por elemento (estimación CPython)	≈ 90 a 120 B (objeto nodo con punteros)	≈ 36 B (referencia + entero)	≈ 100 MB	≈ 360 MB

Lectura de los resultados

- **Búsqueda:** la diferencia medida es de 0,427438 / 0,000022 ≈ 19.400 veces. Como los tamaños son distintos (1 M contra 10 M), conviene normalizar: a igual n = 1 M, el heap tardaría ≈ 0,043 s y la ventaja del AVL seguiría siendo de ≈ 1.900 veces. La causa es estructural: el AVL descarta la mitad de los candidatos en cada paso; el heap no tiene orden lateral y debe recorrer el arreglo.
- **Inserción:** el heap resulta ≈ 13 veces más barato por inserción, porque trabaja sobre un arreglo contiguo y no crea objetos ni rotaciones; el AVL debe crear un nodo y rebalancear. Se aclara que ambos promedios incluyen la lectura y conversión del archivo, y que el AVL se midió con 10 veces menos datos.
- **Eliminación:** ambas son logarítmicas. El heap solo elimina de forma eficiente el máximo (ideal para colas de prioridad); el AVL elimina cualquier valor en O(log n).
- **Memoria:** ambas estructuras son O(n), pero el AVL tiene un factor constante ≈ 3 veces mayor por los objetos y punteros. Con 100 M de nodos el AVL necesitaría ≈ 10 GB, cerca del límite de los 15,4 GB utilizables del equipo y sin margen para el sistema operativo, lo que justifica la muestra de 1 M.

5.2 Algoritmos de ordenamiento

Tabla teórica

Algoritmo	Mejor caso	Caso promedio	Peor caso	Espacio auxiliar	In-place	Estable
Counting Sort	O(n + k)	O(n + k)	O(n + k)	O(k)	No	Sí (versión general)
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	Sí	No
Heap Sort	O(n log n)	O(n log n)	O(n log n)	O(1) (+ pila de recursión O(log n))	Sí	No

Tabla empírica

Algoritmo	n probado	Tiempo medido	Tiempo por elemento	Proyección a 100 M
Counting Sort	100.000.000	71,03 s	0,71 µs	71 s (medido)
Selection Sort	100.000	157,25 s	1.572,5 µs	≈ 5 años

Algoritmo	n probado	Tiempo medido	Tiempo por elemento	Proyección a 100 M
Heap Sort	1.000.000	5,17 s	5,17 µs	—
Heap Sort	10.000.000	58,36 s	5,84 µs	≈ 11 min (teórica)

Comparación a igual n. Con n = 100.000 (el único tamaño con medición de Selection Sort), Heap Sort tardaría ≈ 5,17 s × (10^5 × log2 10^5) / (10^6 × log2 10^6) ≈ 0,43 s (estimación por escalado n log n), unas ≈ 365 veces menos que Selection Sort (157,25 s). La brecha crece con n porque los costos divergen como n^2 contra n log n.

Memoria. Counting Sort necesita ≈ 800 MB para k = 10^8, independientemente de n. Heap Sort y Selection Sort solo necesitan la propia lista de datos (≈ 36 B por elemento) más una cantidad constante de memoria auxiliar. Counting Sort es el más rápido pero el más costoso en memoria; Heap Sort ofrece el mejor equilibrio entre ambos recursos.

6. Conclusiones

1. **Para 100 millones de enteros dentro de un rango conocido, Counting Sort es el más eficiente.** Ordenó el volumen completo en 71 s, con costo lineal O(n + k). Su precio es la memoria (≈ 800 MB) y su restricción de uso: solo funciona con enteros de rango acotado y se vuelve ineficiente si k es mucho mayor que n.
2. **Heap Sort es la mejor opción de propósito general.** Garantiza O(n log n) en todos los casos, ordena in-place con memoria auxiliar mínima y no depende del rango de los valores. Su crecimiento observado (factor 11,29 al decuplicar n) coincide con la teoría. No es estable.
3. **Selection Sort no es viable para grandes volúmenes.** Con solo 100.000 elementos ya tardó 157 s, y su proyección a 100 M es de ≈ 5 años. Su uso se limita a fines didácticos o a entradas muy pequeñas.
4. **El AVL es la estructura adecuada cuando se necesita búsqueda frecuente y aleatoria:** su búsqueda es entre ≈ 1.900 y 19.400 veces más rápida que la del heap, y además mantiene los datos ordenados y permite eliminar cualquier valor en O(log n), a cambio de mayor memoria y de una inserción más lenta.
5. **El Heap es la estructura adecuada para colas de prioridad:** acceso inmediato al máximo, inserción promedio O(1) y extracción en O(log n), con la menor huella de memoria. No debe usarse para búsqueda.
6. **No existe un único ganador:** la elección depende del recurso crítico (tiempo, memoria o tipo de operación) y de las propiedades de los datos. El análisis teórico predice correctamente los resultados medidos, lo que valida el diseño y la implementación.

7. Limitaciones y posibles mejoras

- **Una sola corrida por medición.** Para mayor rigor estadístico conviene repetir cada prueba 3 a 5 veces y reportar el promedio y la dispersión, y medir con time.perf_counter(), de mayor resolución que time.time().
- **Muestras de distinto tamaño.** Se justificaron por costo o por memoria; el análisis se apoyó en tiempos por elemento y en proyecciones teóricas para compararlas.
- **Tiempos con I/O incluida.** Los tiempos de construcción incluyen leer y convertir el archivo; separar el tiempo de I/O del de cómputo daría una medida más pura.
- **Valores repetidos en el AVL.** Con 1 M de valores en un rango de 100 M hay unos miles de duplicados. Para garantizar el balance con duplicados, las condiciones de rotación de la inserción deben basarse en el factor de balance del hijo (como ya se hace en la eliminación) y no en la comparación de valores.
- **Memoria estimada, no medida.** Las cifras de memoria por elemento son estimaciones de CPython de 64 bits; pueden verificarse con tracemalloc o con el Administrador de tareas.
- **Mejoras futuras:** implementar el Min Heap, construir el heap con heapify bottom-up (O(n) garantizado), usar heapify iterativo y comparar contra el ordenamiento incorporado de Python (Timsort) y contra NumPy.

8. Instrucciones de ejecución

1. Instalar Python 3 (no requiere librerías externas; solo módulos de la biblioteca estándar: random, time, os, collections, sys, math).

https://www.python.org/downloads/

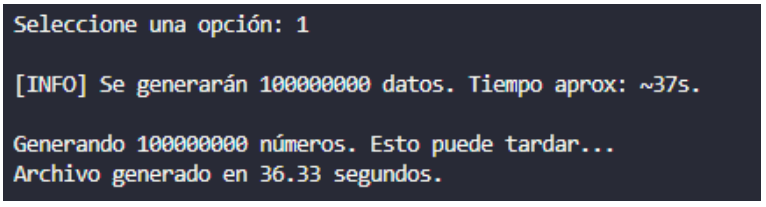
1. Descargar el archivo “tp3.py”, y ejecutarlo con: “python tp3.py”
2. Ejecutar las opciones en este orden: 1 (genera Datos.txt) → 2 → 3 → 4 → 5 → 6 → 7 → 8 (verifica los archivos generados). Las opciones 2 a 7 exigen que Datos.txt exista.
3. Requisitos de recursos: ≈ 1 GB libre en disco para Datos.txt y otro tanto para Datos_CountingSort.txt; ≈ 1 GB de RAM libre para la opción 3 y ≈ 0,5 GB para las opciones 6 y 7 con 10 M de elementos.
4. Las opciones 4, 5, 6 y 7 piden el tamaño de la muestra; los valores por defecto son los usados en este informe.

9. Anexo: capturas de pantalla

Captura	Contenido
1	Especificaciones del equipo y opciones 1, 2 y 3 (generación, máximo/mínimo/frecuencias, Counting Sort)
2	Opciones 4, 5 y 6 (Selection Sort, AVL y Max Heap)
3	Cierre de la opción 6 y opciones 7 y 8 (Heap Sort y verificación)



Opcion 1:



Opcion 2:

```
Seleccione una opción: 2

[INFO] Se procesarán 100000000 datos. Tiempo aprox: ~68s.

Leyendo archivo (esto tomará unos segundos)...
--- Búsqueda finalizada (Tiempo: 65.24s) ---
Número Mayor: 100000000
Número Menor: 1

Los 10 números que más se repiten:
Número: 24357578 - Apariciones: 11
Número: 22757481 - Apariciones: 10
Número: 80920402 - Apariciones: 10
Número: 93382340 - Apariciones: 10
Número: 19673908 - Apariciones: 10
Número: 22292735 - Apariciones: 10
Número: 53121596 - Apariciones: 10
Número: 97528648 - Apariciones: 10
Número: 94254832 - Apariciones: 10
Número: 20034790 - Apariciones: 9
```

Opcion 3:

```
Seleccione una opción: 3

[INFO] Se procesarán 100000000 datos. Tiempo aprox: ~71s.

Iniciando Counting Sort...
Counting Sort finalizado en 69.57 s. Guardado en Datos_CountingSort.txt
```

Opcion 4:

```
Seleccione una opción: 4

[INFO] Tiempo aprox para 100.000 datos: ~157s.
Ingrese el número de entrada (100000 es un valor óptimo para evitar que se cuelgue): 100000

Iniciando Selection Sort con 100000 elementos...
Selection Sort finalizado en 150.24 s. Guardado en Datos_SelectionSort.txt
```

Opcion 5:

```
Seleccione una opción: 5

[INFO] Tiempo aprox para 1.000.000 datos: ~10s.
Ingrese el número de entrada (1000000 es un valor óptimo para evitar que se cuelgue): 1000000

[ÁRBOL AVL] Cargando primeros 1000000 datos...
AVL construido en 9.24 segundos.
Altura del árbol: 24
Cantidad total de nodos: 1000000
Muestra InOrden (primeros 10 elementos): [14, 142, 186, 189, 523, 668, 792, 1109, 1364, 1366]

Ingrese un número para INSERTAR en el AVL: 2026
Insertado. (Tiempo: 0.000034s)
Nueva cantidad total de nodos: 1000001 | Altura: 24

Ingrese un número para BUSCAR en el AVL: 66666
Resultado Búsqueda: NO ENCONTRADO (Tiempo: 0.000019s)
Ingrese un número para ELIMINAR en el AVL: 484845
Operación de eliminación finalizada. (Tiempo: 0.000034s)
Nueva cantidad total de nodos: 1000001
```

Opcion 6:

```
Seleccione una opción: 6

[INFO] Tiempo aprox para 10.000.000 datos: ~8s.
Ingrese el número de entrada (10000000 es un valor óptimo para evitar que se cuelgue): 10000000

[MAX HEAP] Construyendo Heap con primeros 10000000 datos...
Heap construido en 7.90 segundos.
Cantidad de elementos: 10000000
Elemento Raíz (Máximo): 99999990
Altura aproximada del Heap: 23

Ingrese un número para INSERTAR en el Heap: 55555
Insertado. (Tiempo: 0.000007s)
Ingrese un número para BUSCAR en el Heap: 66666
Búsqueda O(n): No encontrado (Tiempo: 0.442907s)
Eliminando el MÁXIMO (Raíz)...
Máximo extraído: 99999990 (Tiempo: 0.000037s)
Nueva Raíz: 99999988
```

Opcion 7:

```
Seleccione una opción: 7

[INFO] Tiempo aprox para 10.000.000 datos: ~58s.
Ingrese el número de entrada (10.000.000 es un valor óptimo para evitar que se cuelgue): 10000000

Iniciando Heap Sort con 10000000 elementos...
Heap Sort finalizado en 69.66 s. Guardado en Datos_HeapSort.txt
```

Opcion 8:

```
Seleccione una opción: 8
Verificación de 'Datos_CountingSort.txt': ORDENADO (Tiempo: 17.81s)
Verificación de 'Datos_SelectionSort.txt': ORDENADO (Tiempo: 0.02s)
Verificación de 'Datos_HeapSort.txt': ORDENADO (Tiempo: 1.74s)
```